

WAFT Development Team

WAFT Comprehensive Feature Showcase

Complete Demonstration of All PDF Generation
Features

Version 1.0

January 11, 2026

Compiled by Feature Showcase Generator

5 Sections | 8 Documents

FEATURE SHOWCASE

Table of Contents

Section 1: Template System

All template types: Field Guide, Lab Notes, Personal Memo, TM Report

Template Field Guide

Template Lab Notes

Template Personal Memo

Template Tm Report

Section 2: Foundation V1

Block-based API with FPDF2

Foundation V1 Showcase

Section 3: Foundation V2

Enhanced blocks with Clinical Standard preset

Foundation V2 Showcase

Section 4: DocumentBuilder

Unified fluent API for PDF generation

DocumentBuilder Showcase

Section 5: Evolution System

Two-page generator with adaptive constraints, metrics, and PNG conversion

Evolution System Showcase

SECTION 1: TEMPLATE SYSTEM

All template types: Field Guide, Lab Notes, Personal Memo, TM Report

SHOWCASE FG-001

WAFT FIELD GUIDE TEMPLATE

Template System Demonstration

FEATURE SHOWCASE

WAFT TEMPLATE SYSTEM: FIELD GUIDE

This document demonstrates the Field Guide template, designed for operational manuals and survival guides.

WARNING

Field guides are critical documents. Follow all procedures exactly as written.

TEMPLATE FEATURES

- ☐ Two-column layout support
- ☐ Warning and caution boxes
- ☐ Equipment checklists
- ☐ Step-by-step procedures
- ☐ Rugged, practical aesthetic

USAGE EXAMPLE

- 1 Use `DocumentBuilder.field_guide()` or `generate_field_guide()`
- 2 Provide HTML content with template-specific classes
- 3 Customize series, number, classification, and metadata

FEATURE SHOWCASE

KEY FEATURES

The Field Guide template provides:

- Professional header/footer with series numbers
- Classification markings
- Warning and caution callout boxes
- Checklist formatting
- Procedure step formatting
- Two-column layout capability

SHOWCASE-LN-001

WAFT LAB NOTES TEMPLATE

PROJECT: Template System Demonstration

RESEARCHER: Feature Showcase Generator

FACILITY: WAFT Development Lab

DATE: 2026-01-11

CLASSIFICATION: FEATURE SHOWCASE

WAFT Template System: Lab Notes

This document demonstrates the Lab Notes template, designed for research documentation and scientific notebooks.

Experiment: Template Feature Analysis

Objective

Document all features of the Lab Notes template system.

Procedure

- 1. Generate document using `generate_lab_notes()`
- 2. Include structured content with headers and lists
- 3. Add metadata and classification

Results

The Lab Notes template provides:

- Scientific notebook aesthetic
- Structured sections for experiments
- Date and metadata tracking
- Professional formatting

Conclusion

The Lab Notes template is ideal for research documentation, experimental records, and scientific notebooks.

I certify that this is an accurate record of my work.

Feature Showcase Generator

2026-01-11

FROM: Feature Showcase Generator

TO: All WAFT Users

DATE: January 11, 2026

RE: Personal Memo Template Features

WAFT Template System: Personal Memo

This document demonstrates the Personal Memo template, designed for staff communications and internal memos.

The Personal Memo template provides:

- Clean, professional memo format
- Header fields for To/From/Subject
- Date and classification support
- Simple, readable layout

Usage

Use `generate_personal_memo()` for internal communications, announcements, and staff memos.

TELEPORT MASSIVE

Making the Impossible, Inevitable™

FEATURE SHOWCASE

SHOWCASE-TM-001

WAFT TM Report Template

Date: January 11, 2026
Author: Feature Showcase Generator
Department: WAFT Development

EXECUTIVE SUMMARY

Template System Demonstration - TM Report format

1. WAFT Template System: TM Report

This document demonstrates the Technical Memo (TM) Report template, designed for corporate reports and technical documentation.

2. Features

- Professional report layout
- Executive summary section
- Structured content organization
- Classification and metadata support

3. Usage

Use `generate_tm_report()` for technical memos, corporate reports, and formal documentation.

This document contains proprietary information of TELEPORT MASSIVE.
Unauthorized disclosure may result in severe penalties.

SECTION 2: FOUNDATION V1

Block-based API with FPDF2

Foundation V1 Block System

This document

demonstrates all

Foundation V1 content

blocks. Foundation V1

uses FPDF2 for pure

Python PDF generation

with a block-based API.

TextBlock

TextBlock is the basic

content block for

paragraphs and text

content. It handles

automatic word wrapping

and page breaks.

KeyVa lueBlock

Technology **EPDF2**

Language **Pure Python**

Status: **Production**

Blocks Available: **6**

LogBlock

2026-01-11 14:00:00 - Document generation started

2026-01-11 14:00:05 - All blocks added

2026-01-11 14:00:10 - PDF rendering complete

WarningBlock

[WARNING]

Foundation V1 requires manual positioning but pro

SignatureBlock

WAFT System: Feature Showcase Generator

January 11, 2026

SECTION 3: FOUNDATION V2

Enhanced blocks with Clinical Standard preset

Foundation V2 Enhanced Blocks

Foundation V2 provides enhanced

typography and professional layout

blocks. This system includes advanced

blocks like MetadataRail, RuleBlock,

and TableBlock.

MetadataRail

Document Metadata

Author: Feature Showcase Generator

Date: 2026-01-11

Classification: FEATURE SHOWCASE

Version: 2.0

TableBlock

Block Type	Description	Status
CoverPage	Professional cover page	Available
MetadataRail	Sidebar metadata display	Available
RuleBlock	Horizontal rule separator	Available
TableBlock	Structured data tables	Available
Enhanced TextBlock	Improved typography	Available

Clinical Standard Preset

The Clinical Standard preset provides

professional scientific documentation

styling with Times New Roman body

text, Helvetica headers, and 1-inch

margins.

SECTION 4:

DOCUMENTBUILDER

Unified fluent API for PDF generation

SHOWCASE DB-001

WAFT DOCUMENTBUILDER SHOWCASE

FOR OFFICIAL USE ONLY

WAFT DOCUMENTBUILDER SYSTEM

The DocumentBuilder provides a unified, fluent API for generating PDFs with WAFT templates.

FEATURES

- Fluent API: Chain methods for readability
- Presets: Common configurations ready to use
- Composition: Build complex documents from simple blocks
- Printer-friendly: One flag, automatic conversion
- Page constraints: exact_pages, max_pages, min_pages with feedback loops

USAGE EXAMPLE

```
from waft import DocumentBuilder

doc = DocumentBuilder.field_guide(
    title="My Guide",
    content="<h2>Introduction</h2><p>Content</p>",
    printer_friendly=True,
    exact_pages=5
)
doc.save("output.pdf")
```

FOR OFFICIAL USE ONLY

PAGE CONSTRAINTS

The DocumentBuilder supports page constraint feedback loops:

- `exact_pages`: Generate exactly N pages
- `max_pages`: Maximum allowed pages
- `min_pages`: Minimum required pages

SECTION 5: EVOLUTION SYSTEM

*Two-page generator with adaptive constraints, metrics, and PNG
conversion*

WAFT Evolution System Showcase

Summary: We decided to implement adaptive constraint enforcement for the two-page generator.

Generated: 2026-01-11 22:53 UTC | Ideas Extracted: 5

What Happened

We decided to implement adaptive constraint enforcement for the two-page generator. The choice was made to use real page counting instead of estimates. We will collect comprehensive metrics for every PDF generation. The final decision was to enable PNG conversion by default for quality verification.

We discovered that character-counting methods are unreliable for page estimation. The key insight is that real measurement beats estimation every time. We learned that adaptive algorithms can achieve exact page counts through iterative adjustment. It turns out that comprehensive metrics enable evolution with quality data.

We need to implement comprehensive testing for all PDF generation features. Next step is to create documentation for the metrics system. We must create example scripts demonstrating each feature. We should build a validation suite that checks all outputs.

Additional Details

The system represents a new approach to document generation with evolutionary algorithms. This is a framework for creating physical knowledge artifacts. The approach combines evolutionary algorithms with constraint satisfaction. The architecture uses genetic material for styling configuration.

How should we handle edge cases in page counting? What is the best approach for handling very long content? Why does the system sometimes need multiple iterations? How can we improve the fitness evaluation algorithm?

Content Breakdown

Type	Count
Decisions	1
Insights	1
Actions	1
Concepts	2
Questions	0